

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Рязанский государственный радиотехнический университет имени В.Ф.
Уткина»
Кафедра «САПР ВС»

УТВЕРЖДАЮ
Руководитель работы
_____/_____
« ____ » _____ 2026 г.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на разработку программного обеспечения
«Библиотека и редактор файлов KiCad на языке Python»
(рабочее название kicadfp)

Этап 1: работа с посадочными местами (корпусами) компонентов

Разработал:
студент группы 346
Хорьков К.А.

Рязань 2026

1 ВВЕДЕНИЕ

1.1 Наименование программы: «Библиотека и редактор файлов KiCad на языке Python», рабочее название kicadfp (далее по тексту Программа).

1.2 Краткая характеристика области применения. Программа предназначена для автоматизированной работы с файлами САПР печатных плат KiCad вне самой САПР: чтение файлов, изменение их содержимого из скриптов и из интерфейса пользователя, сохранение результата в формате, который без ошибок открывается в KiCad. Область применения: учебное и инженерное проектирование печатных плат, подготовка и массовое исправление библиотек компонентов, генерация корпусов по параметрам, автоматизация рутинных операций при ведении библиотек кафедры и личных библиотек разработчика.

1.3 На первом этапе разработки Программа поддерживает один тип сущностей KiCad: посадочные места компонентов (footprints), хранящиеся в файлах с расширением .kicad_mod, и библиотеки посадочных мест в виде каталогов с расширением .pretty. Последующие этапы расширяют Программу на схемы (.kicad_sch), платы (.kicad_pcb), библиотеки символов (.kicad_sym), правила проектирования (.kicad_dru), таблицы библиотек (fp-lib-table, sym-lib-table) и файлы проекта (.kicad_pro). Требования к последующим этапам приведены в разделе 7 в объеме, достаточном для того, чтобы архитектура первого этапа их не исключала.

2 ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

2.1 Основанием для разработки является задание на индивидуальную работу по дисциплине «Автоматизация конструкторского и технологического проектирования» (кафедра САПР ВС РГРТУ), а также практическая потребность, выявленная при выполнении лабораторных работ по САПР KiCad: библиотеки символов и корпусов, схемы и платы для лабораторных работ создавались и правились вручную, при этом однотипные операции (создание корпусов DIP, резисторов, конденсаторов, разъемов с заданным шагом, исправление слоев и размеров площадок, переименование) занимали основное время.

2.2 Тематика работы: инструментальные средства для работы с форматами файлов САПР электронных средств. Условное обозначение темы: kicadfp.

2.3 Заказчик: кафедра САПР ВС РГРТУ. Исполнитель: студент группы 346 Хорьков К.А.

3 НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1 Функциональное назначение. Программа обеспечивает:

- 1) чтение файлов посадочных мест .kicad_mod и библиотек .pretty с построением объектной модели корпуса в памяти;
- 2) просмотр и изменение свойств корпуса, графических элементов, текстовых полей и контактных площадок через программный интерфейс (API на Python), интерфейс командной строки и графический интерфейс пользователя;
- 3) создание новых корпусов, в том числе по параметрам (генераторы типовых корпусов);
- 4) сохранение измененного корпуса в файл .kicad_mod с сохранением совместимости с KiCad и без потери данных, которые Программа не поддерживает напрямую;
- 5) проверку корпуса на формальную корректность перед сохранением.

3.2 Эксплуатационное назначение. Программа используется инженерами-конструкторами, студентами и преподавателями при подготовке библиотек компонентов, в учебном процессе по САПР печатных плат и в скриптах автоматизации. Конечные пользователи: разработчики печатных плат, владеющие KiCad на уровне пользователя, и разработчики скриптов, владеющие Python на базовом уровне.

4 ТРЕБОВАНИЯ К ПРОГРАММЕ

4.1 Требования к функциональным характеристикам

4.1.1 Поддерживаемые форматы. Программа должна читать и записывать файлы посадочных мест в формате S-выражений KiCad версий 6, 7, 8 и 9 (токен footprint, версия формата в поле version). Файлы старого формата PCBnew-LibModule-V1 (библиотеки .mod KiCad версий 4 и ниже) на первом этапе поддерживаются только на чтение с преобразованием в современный формат; запись в старый формат в первый этап не входит. Библиотека .pretty представляет собой каталог, в котором каждый файл .kicad_mod является одним корпусом; Программа должна уметь открывать каталог как библиотеку, перечислять корпуса, добавлять, удалять, переименовывать и копировать корпуса между библиотеками.

4.1.2 Модель данных корпуса. В памяти корпус представляется объектом Footprint со следующими сущностями (полный перечень полей приведен в приложении Б):

- 1) свойства корпуса: имя, версия формата, генератор, слой (F.Cu или B.Cu), описание (descr), ключевые слова (tags), атрибуты (attr: smd, through_hole, board_only, exclude_from_pos_files, exclude_from_bom), поля свойств (property), зазоры и параметры маски и пасты (clearance, solder_mask_margin, solder_paste_margin, solder_paste_ratio), тип зоны

подключения (zone_connect), точка привязки (anchor), уникальные идентификаторы (tstamp или uuid);

- 2) текстовые поля: обозначение (Reference), значение (Value), пользовательские тексты (fp_text user, в новых версиях fp_text и property) с параметрами положение, поворот, слой, размер шрифта, толщина, выравнивание, зеркальность, видимость;
- 3) графические элементы: линия (fp_line), прямоугольник (fp_rect), окружность (fp_circle), дуга (fp_arc), полигон (fp_poly), кривая Безье (fp_curve) с параметрами координаты, слой, ширина линии или параметры stroke, заливка;
- 4) контактные площадки (pad): номер, тип (thru_hole, smd, connect, np_thru_hole), форма (circle, rect, oval, trapezoid, roundrect, custom), координаты и поворот, размер, отверстие (drill: диаметр, овальное отверстие, смещение), слои, параметры формы (roundrect_rratio, chamfer_ratio, chamfer), зазоры и параметры маски и пасты на уровне площадки, зона подключения, номер и имя цепи (net), имя вывода (pinfunction), тип вывода (pintype), для формы custom описание примитивов и опций;
- 5) трехмерные модели (model): путь к файлу, смещение, масштаб, поворот, флаг скрытия;
- 6) зоны и группы внутри корпуса (zone, group) на первом этапе не редактируются, но должны сохраняться без изменений.

4.1.3 Операции редактирования. Программа должна позволять пользователю выполнять следующие действия над открытым корпусом: изменять любое свойство корпуса из перечня п. 4.1.2; добавлять, удалять и изменять текстовые поля; добавлять, удалять и изменять графические элементы, в том числе перемещать, поворачивать, зеркально отражать и менять слой; добавлять, удалять и изменять площадки, в том числе изменять номер, тип, форму, размер, координаты, поворот, отверстие и набор слоев; выполнять групповые операции над набором площадок (перенумерация по правилу, сдвиг, поворот вокруг точки, смена слоев, задание общего размера); генерировать типовые корпуса по параметрам (двухрядные DIP с заданным числом выводов, шагом и расстоянием между рядами, однорядные и двухрядные штыревые разъемы, выводные резисторы и конденсаторы с заданным межцентровым расстоянием, диоды, транзисторы с выводами в линию) с автоматическим построением контура шелкографии и области размещения (F.Fab, F.CrtYd).

4.1.4 Сохранение без потерь (round-trip). Ключевое требование этапа. Программа не должна повреждать структуру файла и не должна удалять данные, которые не поддерживаются напрямую. Для этого файл разбирается в

дерево S-выражений целиком; типизированные объекты модели (площадки, графика, тексты) являются представлениями над узлами этого дерева; узлы неизвестного типа, неизвестные атрибуты известных узлов и порядок узлов сохраняются как есть. При сохранении файла, который открыт и не изменен, результат должен совпадать с исходным по содержанию S-выражений (сравнение по разобранному дереву; допускается различие в пробелах, переводах строк и формате записи чисел). При изменении одного элемента изменяться в файле должен только этот элемент. Числа записываются в формате KiCad: без лишних нулей, с точностью не хуже 0,000001 мм; строки экранируются по правилам KiCad; порядок токенов внутри узла соответствует порядку, который использует KiCad при записи, чтобы файл читался в KiCad и корректно отображался в системах контроля версий.

4.1.5 Проверка корректности. Перед сохранением и по отдельной команде Программа выполняет проверку корпуса и выводит список замечаний с уровнями «ошибка» и «предупреждение»: дублирующиеся номера площадок (с учетом того, что в KiCad одинаковые номера допустимы и означают один вывод, это предупреждение); площадка без слоев; сквозная площадка без отверстия и SMD-площадка с отверстием; отверстие больше площадки; отсутствие обязательных полей Reference и Value; текст Reference или Value не на слое шелкографии или F.Fab; графика на несуществующем слое; недопустимые значения перечислений (тип, форма, слой); нулевые или отрицательные размеры; отсутствие области размещения (CrtYd) как предупреждение. Ошибки блокируют сохранение только по явному указанию пользователя (режим строгой проверки), по умолчанию выводятся как список.

4.1.6 Программный интерфейс (API). Программа оформляется как пакет Python kicadfp с публичным интерфейсом, достаточным для использования без графического интерфейса. Минимальный состав: функции load(path) и loads(text) для чтения, save(footprint, path) и dumps(footprint) для записи, класс Footprint с коллекциями pads, graphics, texts, models и методами доступа к свойствам; классы Pad, Line, Rect, Circle, Arc, Poly, Text, Model с атрибутами, соответствующими токенам формата; класс Library для работы с каталогом .pretty (list, get, add, remove, rename, copy_to); модуль generators с функциями построения типовых корпусов; функция validate(footprint) с возвратом списка замечаний. Все координаты и размеры в API задаются в миллиметрах, углы в градусах. Пример использования API приведен в приложении В.

4.1.7 Интерфейс командной строки. Утилита kicadfp предоставляет команды: info (вывод сводки по корпусу: имя, слой, число площадок, список слоев, габариты), list (перечень корпусов библиотеки), validate, set (изменение свойства по пути вида pad[3].size или footprint.descr), pads (табличный вывод

площадок), gen (генерация типового корпуса с параметрами), fmt (перезапись файла в каноническом форматировании KiCad без изменения содержания), convert (преобразование старого формата .mod в .pretty). Все команды принимают путь к файлу или библиотеке и не изменяют файл без ключа записи или явного указания выходного пути.

4.1.8 Графический интерфейс пользователя. Графический интерфейс реализуется на PySide6 (Qt) и содержит: дерево библиотек и корпусов; панель свойств корпуса; таблицу площадок с редактированием ячеек (номер, тип, форма, X, Y, поворот, размер X, размер Y, отверстие, слой); таблицу графических элементов и текстов; графическое поле предварительного просмотра корпуса с отображением слоев в цветах KiCad, сеткой, масштабированием и выбором элемента щелчком; команды открыть, сохранить, сохранить как, отменить и повторить, проверить, создать типовой корпус. Изменения в таблицах и на поле просмотра применяются к одной и той же модели данных. Графический интерфейс является обязательным для первого этапа, но реализуется после API и командной строки и может быть сдан отдельной версией.

4.1.9 Входные и выходные данные. Входные данные: файлы .kicad_mod, каталоги .pretty, файлы старого формата .mod (только чтение), параметры генераторов из командной строки или файла параметров в формате JSON. Выходные данные: файлы .kicad_mod и каталоги .pretty, отчеты проверки в текстовом виде и в JSON, изображения предварительного просмотра в форматах SVG и PNG. Кодировка всех текстовых файлов UTF-8, перевод строки LF, что соответствует записи самой САПР KiCad.

4.2 Требования к надежности

4.2.1 Программа не должна терять или искажать данные пользователя. Запись файла выполняется через временный файл в том же каталоге с последующим переименованием, чтобы при сбое исходный файл оставался целым. При открытии файла с синтаксической ошибкой Программа сообщает номер строки и позицию ошибки и не создает частично разобранный модель. При обнаружении неизвестных узлов Программа продолжает работу и сохраняет их без изменений.

4.2.2 Все изменения, доступные из графического интерфейса, обратимы (отмена и повтор не менее чем на 100 шагов). Перед закрытием несохраненного корпуса выводится запрос подтверждения.

4.2.3 Программа сопровождается набором автоматических тестов: модульные тесты разбора и записи для каждого типа узла, тесты round-trip на наборе не менее 200 корпусов из стандартных библиотек KiCad (каталоги Package_DIP.pretty, Resistor_THT.pretty, Capacitor_THT.pretty, Connector_PinHeader_2.54mm.pretty, Package_SO.pretty и других), тесты

генераторов и тесты проверки корректности. Критерий приемки: все тесты round-trip проходят, покрытие кода тестами не менее 80 процентов.

4.3 Условия эксплуатации

4.3.1 Программа эксплуатируется на персональном компьютере пользователя в обычных условиях помещения. Специальных требований к квалификации обслуживающего персонала нет; пользователю достаточно навыков работы с KiCad и с командной строкой операционной системы. Для использования API требуется знание Python на уровне написания скриптов.

4.4 Требования к составу и параметрам технических средств

4.4.1 Минимальная конфигурация: процессор с архитектурой x86-64 или ARM64, оперативная память 2 ГБ, свободное место на диске 200 МБ (с учетом Python и Qt), монитор с разрешением не ниже 1280 на 720 точек для графического интерфейса. Программа не требует сетевого подключения.

4.5 Требования к информационной и программной совместимости

4.5.1 Язык реализации Python версии 3.10 и выше. Ядро (разбор, модель, запись, проверка, генераторы) не должно зависеть от сторонних пакетов, кроме стандартной библиотеки Python, чтобы его можно было использовать в скриптах и в среде KiCad (встроенный Python). Графический интерфейс использует PySide6, экспорт изображений использует стандартные средства Qt или пакет cairosvg; эти зависимости объявляются как необязательные (extras) при установке.

4.5.2 Операционные системы: Windows 10 и 11, Linux (дистрибутивы с glibc 2.31 и новее), macOS 12 и новее. Программа распространяется как пакет для установки через pip (wheel и sdist) и как исходный код в репозитории Git с файлами pyproject.toml, README и лицензией.

4.5.3 Совместимость с KiCad. Файлы, записанные Программой, должны без ошибок и предупреждений открываться в редакторе посадочных мест KiCad 8 и 9 и корректно отображаться. Файлы, записанные KiCad 6 и 7, должны читаться Программой. Форматы файлов приняты по официальному описанию KiCad (документация «S-Expression Formats», раздел Footprint) и по фактическому поведению KiCad, проверяемому на файлах стандартных библиотек. Перечень поддерживаемых токенов приведен в приложении А.

4.5.4 Программа не должна использовать внутренний Python API самой KiCad (модуль pcbnew), чтобы работать без установленной САПР. Для проверки совместимости в составе тестов допускается необязательный тест, который при наличии установленного KiCad запускает его командную строку (kicad-cli) для экспорта записанного корпуса и сверки результата.

4.6 Требования к маркировке и упаковке

4.6.1 Программа поставляется в электронном виде: пакет в репозитории и архив с исходным кодом. Маркировка: имя пакета kicadfp, номер версии по схеме семантического версионирования (major.minor.patch), в файле версии и в выводе команды kicadfp --version.

4.7 Требования к транспортированию и хранению

4.7.1 Специальных требований нет. Исходный код и выпуски хранятся в репозитории Git и в резервной копии на носителе заказчика.

5 ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1 В состав программной документации входят: настоящее техническое задание; описание программы (назначение, архитектура, структура пакета, описание модели данных и форматов); руководство пользователя (установка, работа с командной строкой, работа с графическим интерфейсом, примеры); руководство программиста (описание API с примерами, правила расширения на новые типы файлов); программа и методика испытаний (перечень тестов, порядок запуска, критерии); текст программы (исходный код в репозитории с комментариями и строками документации).

5.2 Документация оформляется в электронном виде в форматах Markdown (в репозитории) и DOCX или PDF (для сдачи), пояснительная записка оформляется по ГОСТ 7.32-2017.

6 ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1 Ориентировочная экономическая эффективность выражается в сокращении времени на подготовку и исправление библиотек корпусов. По опыту выполнения лабораторных работ ручное создание одного корпуса типа DIP14 в редакторе модулей KiCad занимает от 10 до 15 минут, создание библиотеки из 8 корпусов около полутора часов; с генераторами и групповыми операциями Программы та же библиотека создается за несколько минут, а массовое исправление (например, смена размера площадок во всех корпусах библиотеки) выполняется одной командой вместо ручного редактирования каждого файла.

6.2 Предполагаемая потребность: учебные группы кафедры при выполнении лабораторных и курсовых работ по САПР печатных плат, а также разработчики, ведущие собственные библиотеки. Аналоги: скрипты на встроенном Python KiCad (требуют установленной САПР и привязаны к ее версии), библиотека kiutils (поддерживает форматы, но не имеет графического интерфейса и генераторов), онлайн-генераторы корпусов (не работают с существующими файлами). Преимущество Программы в сочетании

независимого ядра, сохранения без потерь, генераторов и интерфейса пользователя.

7 СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

7.1 Разработка выполняется по стадиям ГОСТ 19.102-77. Содержание этапов первого этапа (посадочные места) приведено в таблице 1.

Таблица 1 – Стадии и этапы разработки

Этап	Содержание работ	Результат	Срок
1. Техническое задание	Уточнение требований, изучение формата .kicad_mod на файлах стандартных библиотек KiCad 6, 7, 8, 9, согласование ТЗ	Утвержденное ТЗ	2 недели
2. Эскизный проект	Выбор архитектуры (дерево S-выражений плюс типизированные представления), проектирование модели данных и API, макет графического интерфейса	Описание архитектуры, схема модели данных, макеты окон	2 недели
3. Технический проект	Детальное проектирование разборщика и записи, правил форматирования, алгоритмов проверки и генераторов, плана тестирования	Технический проект, программа и методика испытаний	2 недели
4. Рабочий проект, ядро	Реализация разборщика и записи S-выражений, модели корпуса, round-trip, проверки, генераторов, командной строки; модульные тесты и тесты round-trip	Пакет kicadfr версии 0.1 с API и CLI, отчет о тестировании	5 недель
5. Рабочий проект, интерфейс	Реализация графического интерфейса, предварительного просмотра, отмены и повтора; интеграционные тесты	Пакет kicadfr версии 0.2 с GUI	4 недели
6. Испытания и внедрение	Приемочные испытания по программе и методике, проверка в KiCad 8 и 9, исправление замечаний, подготовка документации и выпуск	Версия 1.0, комплект документации	2 недели

7.2 Последующие этапы разработки (вне сроков настоящего ТЗ, требования фиксируются для обеспечения расширяемости архитектуры):

- 1) этап 2: библиотеки символов .kicad_sym (символы, секции, выводы, графика, свойства) и таблицы библиотек sym-lib-table, fp-lib-table;
- 2) этап 3: схемы .kicad_sch (компоненты, провода, шины, метки, иерархические листы, символы питания, флаги «не соединено»), формирование списка цепей;
- 3) этап 4: платы .kicad_pcb (модули с привязкой к корпусам библиотеки, дорожки, переходные отверстия, зоны, контур платы, классы цепей, настройки), подсчет длин цепей, экспорт Gerber через kicad-cli;

- 4) этап 5: правила проектирования .kicad_dru и файлы проекта .kicad_pro;
- 5) этап 6: поддержка старых форматов (EESchema v2 .sch и .lib, PCBnew-BOARD .brd, .mod) для чтения и преобразования в современные форматы.

7.3 Архитектурные требования, вытекающие из последующих этапов: разборщик и запись S-выражений выделяются в отдельный модуль без привязки к типу файла; модель каждого типа файла строится как слой представлений над общим деревом; правила форматирования (порядок токенов, перенос строк, формат чисел) описываются таблицей на тип узла и дополняются для новых типов без изменения ядра; проверки оформляются как набор независимых правил с общим интерфейсом.

8 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ

8.1 Виды испытаний: автоматические тесты (модульные, round-trip, генераторы, проверка) при каждом изменении кода; приемочные испытания по программе и методике испытаний в присутствии заказчика.

8.2 Приемочные испытания включают: открытие и сохранение без изменений не менее 200 корпусов стандартных библиотек KiCad с последующим сравнением деревьев S-выражений (совпадение 100 процентов); открытие сохраненных файлов в KiCad 8 и 9 без сообщений об ошибках; выполнение сценариев редактирования из приложения Г с проверкой результата в KiCad; генерацию корпусов dip14, mlt, snp8 из лабораторных работ и сравнение с корпусами, созданными вручную; проверку, что файл с искусственно добавленным неизвестным узлом после открытия, изменения одной площадки и сохранения содержит этот узел без изменений.

8.3 Программа считается принятой, если все автоматические тесты проходят, все сценарии приемочных испытаний выполнены без ошибок, документация представлена в полном объеме.

ПРИЛОЖЕНИЕ А. СТРУКТУРА ФАЙЛА .KICAD_MOD

А.1 Файл посадочного места представляет собой одно S-выражение с корневым узлом footprint. Пример минимального корпуса резистора с двумя сквозными площадками, записанного KiCad 8:

```
(footprint "R_THT_10mm" (version 20240108) (generator "pcbnew")
  (layer "F.Cu")
  (descr "Resistor, pitch 10 mm")
  (tags "resistor")
  (attr through_hole)
  (property "Reference" "REF**" (at 5 -2.5 0) (layer "F.Silks")
    (uuid "...") (effects (font (size 1 1) (thickness 0.15))))
  (property "Value" "R_THT_10mm" (at 5 2.5 0) (layer "F.Fab")
    (uuid "...") (effects (font (size 1 1) (thickness 0.15))))
  (fp_line (start 1.25 -1.25) (end 8.75 -1.25)
    (stroke (width 0.12) (type solid)) (layer "F.Silks") (uuid
  "..."))
  (fp_rect (start -1 -2) (end 11 2)
    (stroke (width 0.05) (type solid)) (fill none) (layer
  "F.CrtYd") (uuid "..."))
  (pad "1" thru_hole rect (at 0 0) (size 1.3 1.3) (drill 0.8)
    (layers "*.Cu" "*.Mask") (uuid "..."))
  (pad "2" thru_hole circle (at 10 0) (size 1.3 1.3) (drill 0.8)
    (layers "*.Cu" "*.Mask") (uuid "..."))
  (model "${KICAD8_3DMODEL_DIR}/Resistor_THT.3dshapes/R.wrl"
    (offset (xyz 0 0 0)) (scale (xyz 1 1 1)) (rotate (xyz 0 0
  0)))
)
```

А.2 Перечень токенов, которые Программа обязана распознавать и редактировать на первом этапе, приведен в таблице А.1. Все прочие токены сохраняются без изменений.

Таблица А.1 – Поддерживаемые токены формата

Узел	Дочерние токены	Примечание
footprint	version, generator, generator_version, locked, placed, layer, tedit, uuid или tstamp, descr, tags, property, path, autoplace_cost90, autoplace_cost180, solder_mask_margin, solder_paste_margin, solder_paste_ratio, clearance, zone_connect, thermal_width, thermal_gap, attr, private_layers, net_tie_pad_groups	Корневой узел, имя в первом аргументе
fp_text, property	reference или value или user, текст, at (x y угол), unlocked, layer, hide, uuid, effects (font size thickness bold italic, justify, mirror)	В KiCad 8 Reference и Value записаны как property, в KiCad 6 и 7 как fp_text
fp_line, fp_rect, fp_circle, fp_arc, fp_poly, fp_curve	start, end, center, mid, pts (xy ...), width или stroke (width type), fill, layer, locked, uuid	Толщина в KiCad 6 задается токеном width, в 7 и новее токеном stroke
pad	номер, тип, форма, at, size, drill (диаметр, oval, offset), layers, property, remove_unused_layers, keep_end_layers, roundrect_rratio, chamfer_ratio, chamfer, net, pinfunction, pintype, die_length,	Для формы custom примитивы gr_line, gr_arc, gr_circle, gr_rect, gr_poly внутри

Узел	Дочерние токены	Примечание
	solder_mask_margin, solder_paste_margin, solder_paste_margin_ratio, clearance, zone_connect, thermal_bridge_width, thermal_gap, options, primitives, uuid	primitives
model	путь, hide, opacity, offset (xyz), scale (xyz), rotate (xyz)	Пути с переменными окружения KiCad сохраняются как строки
zone, group	все дочерние	Не редактируются, сохраняются

А.3 Правила записи. Строки заключаются в двойные кавычки, символы кавычки и обратной косой черты экранируются; числа записываются без экспоненты, дробная часть без завершающих нулей, целые без точки; каждый дочерний узел верхнего уровня записывается с новой строки с отступом в два пробела; узлы pad, property и графика записываются в порядке их следования в исходном файле, новые элементы добавляются в конец соответствующей группы; идентификаторы uuid для новых элементов генерируются по UUID версии 4.

ПРИЛОЖЕНИЕ Б. МОДЕЛЬ ДАННЫХ

Б.1 Классы модели и их основные атрибуты приведены в таблице Б.1. Все классы хранят ссылку на узел дерева S-выражений и изменяют его при изменении атрибутов, поэтому неизвестные данные в узле сохраняются.

Таблица Б.1 – Классы модели корпуса

Класс	Атрибуты	Методы
Footprint	name, version, generator, layer, descr, tags, attrs (set), properties (dict), clearance, solder_mask_margin, solder_paste_margin, solder_paste_ratio, zone_connect, uuid, pads (list), graphics (list), texts (list), models (list), unknown (list узлов)	pad(number), bbox(), move(dx, dy), rotate(angle, origin), flip(), renumber_pads(rule), validate(), to_sexpr(), copy()
Pad	number, type, shape, x, y, angle, size_x, size_y, drill (Drill или None), layers (list), roundrect_rratio, chamfer_ratio, chamfer (list), net (номер, имя), pinfunction, pintype, clearance, solder_mask_margin, solder_paste_margin, zone_connect, uuid, primitives (list)	move, rotate, set_layers(preset), is_tht(), is_smd()
Drill	diameter, oval (bool), width, offset_x, offset_y	
Line, Rect, Circle, Arc, Poly, Curve	координаты по типу (start, end, center, mid, points), layer, width, stroke_type, fill, uuid	move, rotate, mirror, length()
Text	kind (reference, value, user), text, x, y, angle, layer, hide, font_size_x, font_size_y, thickness, bold, italic, justify, mirror, uuid	move, rotate
Model	path, hide, opacity, offset, scale, rotate	
Library	path, names (list)	get(name), add(footprint), remove(name), rename(old, new), copy_to(other, name), save_all()
Issue	level (error, warning), code, message, element	

Б.2 Единицы: миллиметры и градусы. Система координат совпадает с KiCad (ось Y направлена вниз). Слои задаются строками KiCad (F.Cu, B.Cu, F.SilkS, B.SilkS, F.Mask, B.Mask, F.Paste, B.Paste, F.Fab, B.Fab, F.CrtYd, B.CrtYd, Edge.Cuts, User.N и другие) и групповыми обозначениями со звездочкой (*.Cu, *.Mask); Программа хранит их как строки и проверяет по списку допустимых значений.

ПРИЛОЖЕНИЕ В. ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

В.1 Использование API: открыть корпус, увеличить все площадки, переименовать, сохранить в другую библиотеку.

```
import kicadfp

fp = kicadfp.load("Package_DIP.pretty/DIP-14_W7.62mm.kicad_mod")
for pad in fp.pads:
    pad.size_x = pad.size_y = 1.3
    pad.drill.diameter = 0.8
fp.pad("1").shape = "rect"
fp.name = "dip14"
fp.descr = "DIP-14, шаг 2,5 мм, ряды 7,5 мм (лабораторная работа)"
for issue in fp.validate():
    print(issue.level, issue.code, issue.message)
lib = kicadfp.Library("br8_timer.pretty")
lib.add(fp)
lib.save_all()
```

В.2 Генерация типового корпуса: двухрядный разъем 2 на 4 с шагом 2,5 мм и расстоянием между рядами 5 мм, с двумя монтажными отверстиями.

```
from kicadfp.generators import pin_header

fp = pin_header(rows=2, cols=4, pitch=2.5, row_pitch=5.0,
                pad_size=1.3, drill=0.8, first_square=True,
                mounting_holes=[(11.25, -5.0, 3.0), (11.25, 15.0,
3.0)],
                name="snp8")
kicadfp.save(fp, "snp8.kicad_mod")
```

В.3 Командная строка: сводка, изменение свойства, проверка, генерация.

```
kicadfp info dip14.kicad_mod
kicadfp pads dip14.kicad_mod
kicadfp set dip14.kicad_mod "pad[*].drill" 0.8 --write
kicadfp set dip14.kicad_mod footprint.descr "DIP-14" -o
dip14_new.kicad_mod
kicadfp validate br8_timer.pretty --strict
kicadfp gen dip --pins 14 --pitch 2.5 --row-pitch 7.5 -o
br8_timer.pretty/dip14.kicad_mod
kicadfp fmt br8_timer.pretty
kicadfp convert old_lib.mod -o old_lib.pretty
```

ПРИЛОЖЕНИЕ Г. СЦЕНАРИИ ПРИЕМОЧНЫХ ИСПЫТАНИЙ

Таблица Г.1 – Сценарии испытаний

Номер	Сценарий	Ожидаемый результат
1	Открыть 200 корпусов из стандартных библиотек KiCad 8, сохранить без изменений, сравнить деревья S-выражений	Совпадение деревьев для всех файлов
2	Открыть корпус KiCad 6 (толщина линий токеном width, тексты fp_text), сохранить, открыть в KiCad 8 и 9	Файл открывается без ошибок, графика и тексты на месте
3	В корпус DIP-14 добавить неизвестный узел (example_token 1 2), изменить размер площадки 1, сохранить	Неизвестный узел сохранен без изменений, изменена только площадка 1
4	Изменить у всех площадок форму на roundrect с ratio 0,25, слои на F.Cu F.Mask F.Paste, тип на smd, удалить отверстия	Проверка не выдает ошибок, KiCad показывает SMD-корпус
5	Сгенерировать dip14, mlt, snp8 по параметрам лабораторной работы, открыть в KiCad	Площадки 1,3 мм, отверстия 0,8 мм, первая площадка квадратная, шаг и контур соответствуют заданию
6	Задать площадке отверстие 1,5 мм при размере 1,3 мм, выполнить проверку	Сообщение об ошибке «отверстие больше площадки»
7	Открыть файл с синтаксической ошибкой (незакрытая скобка)	Сообщение с номером строки и позиции, модель не создается
8	В графическом интерфейсе переместить площадку мышью и в таблице, отменить, повторить, сохранить	Значения в таблице и на поле просмотра совпадают, файл содержит итоговые координаты
9	Скопировать корпус из одной библиотеки .pretty в другую с переименованием	В целевой библиотеке появился файл с новым именем и полем name, исходный не изменен
10	Преобразовать библиотеку старого формата .mod (PCBnew-LibModule-V1) в .pretty	Каждый модуль сохранен отдельным файлом .kicad_mod, единицы переведены из децимил в миллиметры, открывается в KiCad